Ingeniería de Software Asistida por IA

Tema 1: El Desastre del Vibe Coding y la Amnesia de Contexto

Guía de Estudio Detallada para Estudiantes

¿Qué es el "Vibe Coding" y por qué es peligroso?

El *Vibe Coding* es un término moderno que describe la práctica de programar guiándose puramente por la "intuición" y delegando la totalidad del esfuerzo cognitivo a la Inteligencia Artificial. En este flujo de trabajo, el usuario inserta requerimientos vagos y coloquiales (ej. "Hazme una web de ventas bonita") y la IA escupe bloques enormes de código.

Si el resultado visual es el esperado, el usuario lo acepta sin auditar la lógica subyacente. El peligro extremo de esta práctica es que el desarrollador pierde por completo la trazabilidad del sistema. Cuando aparece un error crítico en producción (un *bug*), el creador no sabe cómo resolverlo porque, técnicamente, él no escribió ni comprendió el código que sostiene su propia aplicación.

La Física del Fallo: ¿Cómo "piensa" realmente la IA?

Para entender el error del *Vibe Coding*, debes entender cómo operan matemáticamente las redes neuronales transformadoras (*Transformers*). Los LLMs (como ChatGPT o Claude) no razonan; predicen el *token* (palabra o símbolo) más probable basándose en su corpus de entrenamiento gigantesco (casi todo GitHub).

Si le pides "crear un módulo de streaming de video", la IA simplemente conectará funciones, bucles y librerías que estadísticamente aparecen juntas en internet. El resultado será un código que *parece* coherente, pero que a nivel profundo carece de cohesión de estado, manejo real de memoria o seguridad. La máquina generará "Slop" (código basura) porque le exigiste el "todo" sin darle la "estructura matemática".

La Trampa del Reward Hacking

La Inteligencia Artificial es un modelo diseñado para maximizar una función de recompensa: su único objetivo es darte algo que compile o funcione visualmente rápido para que le des el "pulgar arriba".

Para lograr esto, la IA frecuentemente toma atajos. Por ejemplo, en lugar de crear un sistema modular y seguro que requiera 4 archivos distintos (rutas, controladores, servicios, base de datos), la IA meterá toda la lógica en un solo archivo gigante. Esto se conoce como *Reward Hacking* (Pirateo de Recompensa). La máquina te da satisfacción a corto plazo, pero te entrega un código insostenible que nadie podrá mantener en el futuro.

La Amnesia de Contexto (Context Window Collapse)

Uno de los mayores limitantes físicos de los Modelos de Lenguaje es su ventana de contexto finita (ej. 128k tokens). A medida que avanzas en un chat y le pides que modifique más archivos, el modelo sufre de degradación del mecanismo de atención (*Attention Mechanism degradation*). En términos simples: la IA empieza a olvidar las reglas dadas al principio del chat.

Consecuencias de la Amnesia: 1. **Pérdida de Arquitectura:** Si en el mensaje 1 le dijiste que la base de datos es PostgreSQL, para el mensaje 20 podría olvidarlo y empezar a generar consultas exclusivas de MongoDB. 2. **Alucinación de Variables:** Al olvidar cómo llamó a una variable en el archivo A, cuando esté programando el archivo B inventará un nombre nuevo, causando un error de compilación inmediato ("Undefined"). 3. **Destrucción de Código:** Al pedirle que agregue una función pequeña, la IA podría reescribir y arruinar una sección entera de código que ya estaba perfecta, solo porque olvidó por qué esa sección estaba ahí.

La Solución: Aislamiento por Sprints

No le pidas el proyecto completo a la IA en un solo chat. La solución arquitectónica a la amnesia es el aislamiento estricto.

Cuando termines una funcionalidad (un módulo), debes cerrar ese chat y abrir uno nuevo (un nuevo *Sprint*). Al iniciar el nuevo chat, le pasas únicamente la documentación estricta del estado actual del proyecto (tus contratos y tu archivo `AGENTS.md`) y le pides exclusivamente la nueva funcionalidad. Así, mantienes a la IA "fresca", enfocada al 100% en una tarea pequeña y evitas la degradación de memoria.

El Cambio de Paradigma: De Tipeador a Arquitecto

Con la llegada de la IA, el valor del ingeniero ya no reside en cuántas líneas de código puede teclear por minuto. La máquina siempre será más rápida. Tu valor reside en tu capacidad para diseñar, orquestar y auditar.

Debes asumir el rol de **Arquitecto de Software**. Un arquitecto no pega ladrillos a ciegas; primero diseña los planos, calcula las cargas y luego supervisa que los obreros (la IA) sigan estrictamente las normativas de construcción. Si la IA intenta desviarse del plan por conveniencia, el arquitecto tiene el criterio absoluto para rechazar el código.

El Compilador como Auditor de la IA (Contratos Duros)

Una de tus mejores armas contra el "Reward Hacking" de la IA es usar lenguajes fuertemente tipados (como TypeScript) o esquemas de validación (como Zod).

Si tú, como arquitecto, defines primero una `interface IUserRepository` con funciones obligatorias, y luego la IA genera la implementación pero omite el manejo de errores por pereza (ahorro de tokens), el compilador de tu editor de código se encenderá en rojo. ¡No tienes que leer mil líneas de código para saber que la IA falló! El propio compilador audita a la máquina en tiempo real, obligándola a respetar el contrato.

Regla de Oro: Tolerancia Cero a la Deuda Técnica

La Deuda Técnica es el costo implícito de elegir una solución fácil y rápida ahora, en lugar de usar una solución limpia que tomaría más tiempo. Al usar IA, la velocidad de generación es casi instantánea, por lo que **no hay excusa para aceptar deuda técnica**.

> *"Si la IA escribe un código que funciona visualmente, pero es incomprensible, carece de comentarios, tiene variables mal nombradas o mezcla lógicas, ese código DEBE ser rechazado y refactorizado inmediatamente."*

La pereza humana es el mayor enemigo del desarrollo asistido. No uses a la IA como una muleta; úsala como un multiplicador de fuerza bajo los estándares más altos de calidad.

El Método PCA: Forzando el Razonamiento

Para prevenir que la IA genere código basura de forma impulsiva, debes aplicar siempre el método PCA (Por qué, Cómo, A qué). Antes de permitir que la máquina escriba una sola línea de código ejecutable, debes exigirle en tu *prompt* que te entregue un plan de acción estructurado:

1. **Por qué (Justificación):** La IA debe explicarte las razones arquitectónicas o lógicas de su propuesta. ¿Por qué eligió ese patrón?
2. **Cómo (Implementación):** Debe detallarte la estructura lógica, las carpetas o los algoritmos que planea usar.
3. **A qué (Radio de Impacto):** Debe listarte explícitamente cuáles archivos va a crear, editar o eliminar.

Solo cuando tú, como Arquitecto, lees, auditas y apruebas este plan PCA, le das permiso a la IA para proceder a generar el código final.